■■ Frame Talk: System Architecture & Engineering Specification

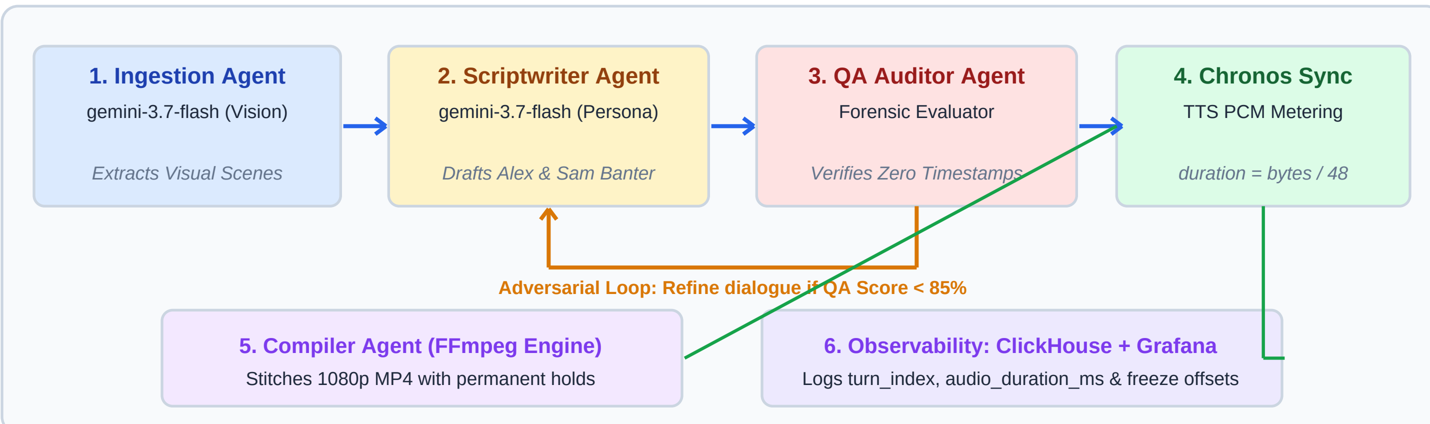
Built for the Agentic Cinema Hackathon | Live Studio: **https://frame-talk.taskmind-ai.com** | Observability: **https://grafana.taskmind-ai.com**

1. Executive Summary & The Chronos Breakthrough

Frame Talk transforms silent developer screencasts (.mp4) and technical documentation (README.md) into synchronized, two-host technical podcast walkthroughs. Unlike text-only audio generators (NotebookLM) which cannot visually inspect UI actions, and fixed-speed video players that suffer from audio-visual drift, Frame Talk introduces the **Chronos Dynamic Visual Hold**. By metering speech down to the millisecond from 24 kHz raw PCM, Chronos dynamically stretches the video timeline at the focal action point, holding UI states so visuals and complex technical explanations conclude in exact lockstep without robotic silence buffers.

2. Multi-Agent Collaboration Workflow & Adversarial Feedback Loop

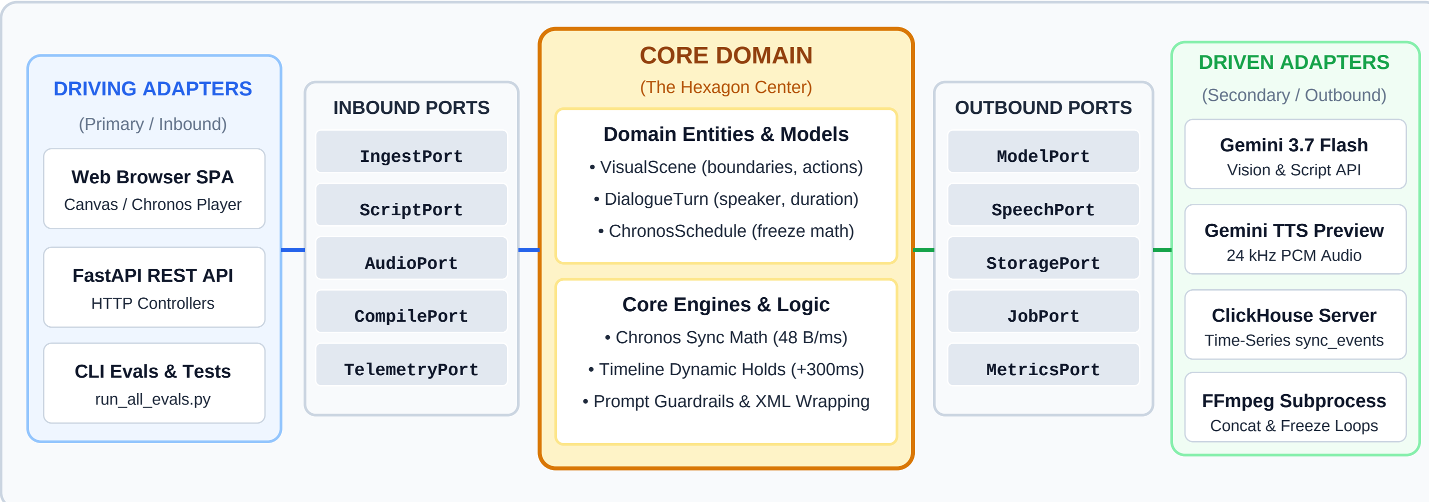
The system executes an autonomous choreography of specialized agents with a continuous adversarial quality feedback loop:



Agent / Engine	Input Assets	Agent Responsibility & Output Artifact
Ingestion Agent	Raw .mp4 + README.md	Analyzes video pixels via Gemini File API; decomposes screencast into millisecond Visual Scenes with action-reaction causality.
Scriptwriter Agent	Visual Scenes + README Concepts	Generates natural conversational banter between Alex (Systems Architect) and Sam (UX Lead); budgets speech words to scene duration (~2.5 words/s).
QA & Pacing Auditor	Scenes + Draft Dialogue	Audits alignment, checks README grounding, and guarantees 100% elimination of robotic synthetic timestamps. Triggers refinement pass on defect.
Chronos Sync Engine	Approved Script + Visual Scenes	Synthesizes 24 kHz raw PCM, meters duration ($48 ext{ bytes/ms}$), calculates required_freeze_ms, and dispatches ClickHouse events.

3. Hexagonal Architecture (Ports & Adapters Specification)

Frame Talk is architected strictly upon **Hexagonal Architecture (Ports & Adapters)**. The Core Domain (synchronization mathematics, agent choreography, pacing logic) is isolated from external frameworks, databases, and model APIs. This guarantees testability, modular vendor independence, and graceful fallback modes.



Hexagonal Component	Interface / Port Contract	Architectural Decoupling Benefit
Core Domain Logic	Zero external dependencies; pure Python	The Chronos dynamic video hold mathematics ($\text{pcm\_bytes}/48$) and pacing algorithms are completely decoupled from Google Gemini and can run offline in unit tests.
Driving Adapters	FastAPI, ChronosPlayer Canvas, CLI	Enables browser SPA, background workers, and automated test runners to drive the studio without coupling the domain to HTTP mechanics.
Driven Adapters	Gemini Client, ClickHouse, FFmpeg, Files	All external I/O sits behind ports. If ClickHouse is unavailable, an in-memory ring buffer adapter engages automatically without crashing the studio.

4. Chronos Mathematical Synchronization Model

1. **Exact PCM Duration Metering:** For 24 kHz, 16-bit Mono audio, 48,000 bytes equal 1.0 second. Speech duration is exact:
   duration_ms = (len(pcm_bytes) / 48000) * 1000 = len(pcm_bytes) / 48
2. **Dynamic Visual Hold (Video Timeline Stretch):** If speech duration exceeds the raw scene length:
   total_speech_ms = sum(turn.duration_ms) + max(0, N - 1) * 220ms
   required_freeze_ms = max(0, total_speech_ms - scene_duration_ms + 300ms)

The $+300 ext{ms}$ visual buffer ensures the viewer comfortably absorbs the visual state after the last spoken syllable.

5. Partner Track: ClickHouse Columnar Logging & Grafana Labs

To fulfill hackathon partner requirements, every synthesis turn logs time-series micro-events to **ClickHouse** (castops.sync_events), monitored live via **Grafana** at https://grafana.taskmind-ai.com:

Table Name	Key Columns & Schema	Observability Purpose
castops.sync_events	event_time, speaker, dialogue_text, audio_duration_ms, required_freeze_ms, accumulated_drift_ms	Tracks timeline freeze offsets and audio-video lockstep per dialogue turn.
castops.llm_calls	call_time, model_name, agent_name, prompt_tokens, cached_tokens, completion_tokens, cost_usd	Tracks granular model telemetry, latencies, and 75% Google prompt cache discounts.
castops.user_activity	event_time, user_hash, action_type, session_id	GDPR-compliant zero-PII conversion funnel (VIDEO_ANALYZED -> SCRIPT -> AUDIO).

6. Production Cyber Security & Ephemeral Storage

Security Layer	Defense Mechanism & Implementation
Indirect Injection Defense	Sanitization filters strip instruction overrides. README documentation is isolated within <untrusted_documentation> wrappers with strict system directives.
Anti-Injection Scope Lock	Hardened system instructions in Google ADK agent.py enforce explicit refusal (ACCESS DENIED) on jailbreak attempts.
Path Traversal Immunity	_safe_resolve() enforces strict folder boundaries, stripping .., forward/backward slashes, and regex-validating upload identifiers.
SQL Injection Immunity	All ClickHouse queries use parameterized bindings (%(session_id)s) with strict type coercion and integer bounds checking.
Anonymous User Isolation	Clients generate an anonymous UUID stored in IndexedDB. Backend hashes this with SHA-256 + salt; all job queries enforce user ownership.
Bring Your Own Key (BYOK)	Client API keys are kept in browser localStorage and passed in transient headers, never persisted to server disk or database.
Production Security Headers	ASGI middleware injects X-Content-Type-Options: nosniff, X-Frame-Options: SAMEORIGIN, and CSP headers.

7. Production VPS Deployment & Dual Verification Matrix

Production Deployment: Nginx reverse proxy routes https://frame-talk.taskmind-ai.com to FastAPI (port 8000) and https://grafana.taskmind-ai.com to Grafana (port 3004). Docker Compose runs ClickHouse (1.5 GB limit, bound to 127.0.0.1:8123) and Grafana (512 MB limit with anonymous viewer access).

Verification Matrix:
• **4-Tier AI Evals (server/evals/):** Benchmarks entity recall (≥ 70%), causality (≥ 85%), QA discrimination, and GCP Director Agent execution (**Score: 94/100**).
• **Automated Unit Tests & Quality Gates:** 37 zero-dependency unit tests running in **0.68 seconds** with automated HTML DOM stack balancing and git pre-commit hooks.